

Autodesk RAG Chatbot

Built by Asad Molayari — Senior ML Engineer Take-Home Assessment

A production-quality Retrieval-Augmented Generation chatbot for Autodesk product documentation.

This system ingests 1,218 noisy HTML pages from the Autodesk website, builds a hybrid retrieval index with two chunking strategies, and provides grounded conversational answers with full source citation transparency. The entire stack is open-source and runs locally — no external API keys required.

Data & Artifacts

Source code: <https://github.com/molayariasad/autodesk-rag-chatbot>

The HTML corpus is provided separately by Autodesk. After cloning the repo, place the HTML files in `data/raw/` before running ingestion:

```
data/raw/                ← Place the 1,218 HTML files here
before ingesting
data/chroma_db/          ← Placeholder in repo – populated
                          automatically by ingestion
data/processed/experiment_results/ ← 11 experiment JSON results (included
in repo)
data/processed/trulens.sqlite ← TruLens evaluation history (included
in repo)
```

Quick Start

Option 1: Docker (Recommended for Windows/Linux users)

```
# 1. after uploading the .html data to data/raw/
# 2. Launch everything (Ollama + FastAPI + Streamlit)
docker-compose up --build

# 3. Access
open http://localhost:8501      # Streamlit Chat UI
open http://localhost:8000/docs # FastAPI Swagger Docs
```

First run pulls `gemma3:4b` (~4GB) and runs ingestion (~60s). Subsequent runs start in ~15s.

Option 2: Local Development (Recommended for Apple Silicon users)

```
# Prerequisites: Python 3.12+, Ollama installed (https://ollama.ai)

# 1. Install dependencies (uv sync creates .venv and installs everything
# automatically)
pip install uv
uv sync

# 2. Pull the LLM (gemma3:4b – optimised for Apple Silicon Metal GPU and
# can work on Windows machine too)
ollama pull gemma3:4b
# Memory constrained (<4GB free)? Use: ollama pull llama3.2:3b-instruct-
q4_K_M

# 3. Configure environment
cp .env.example .env # defaults work out of the box for local Ollama

# 4. Building index from scratch. make sure you have already done cp
/path/to/html/files/*.html data/raw/
uv run python scripts/ingest.py --data-dir ./data/raw

# Note: You need 3–4 terminal windows running simultaneously:
#   Terminal 1: ollama serve
#   Terminal 2: uvicorn (API)
#   Terminal 3: streamlit (UI)
#   Terminal 4: run_admin_dashboard.py (optional, TruLens)

# 5. Start Ollama in a dedicated terminal – keep it running
ollama serve

# 6. Start the API (new terminal)
uv run uvicorn src.api.main:app --port 8000

# 7. Start the UI (new terminal)
uv run streamlit run ui/app.py

# 8. (Optional) TruLens admin dashboard (new terminal)
uv run python scripts/run_admin_dashboard.py # → http://localhost:8502
```

⚠ Platform Performance Notes

Platform	Recommended method	LLM speed
Mac Apple Silicon (M1/M2/M3/M4)	Local dev (ollama serve + uvicorn)	30–60 tok/s (Metal GPU)
Windows	Docker (docker-compose up) or local dev	5–15 tok/s (CPU) — slower than Mac
Linux	Docker or local dev	Fast (native CPU or GPU)

Note on Windows performance: `gemma3:4b` is optimised for Apple Silicon Metal GPU and runs significantly faster on Mac. On Windows, inference runs on CPU at 5–15 tok/s — expect couple minutes per response. This is expected behaviour, not a bug. For faster responses on Windows, switch to `llama3.2:3b` via `LLM_MODEL=llama3.2:3b` in `.env`.

⚠ Important: Docker and Local Development are mutually exclusive

Both use port `11434` for Ollama. Running both simultaneously causes a port conflict.

Mode	Ollama	Ports used
Local dev	<code>ollama serve</code> in terminal	11434, 8000, 8501
Docker	Managed automatically by docker-compose	11434, 8000, 8501

Before switching modes:

```
# Switching from local → Docker: stop Ollama first
kill ollama # or quit the Ollama menu bar app

# Switching from Docker → local: stop Docker first
docker-compose down
ollama serve
```

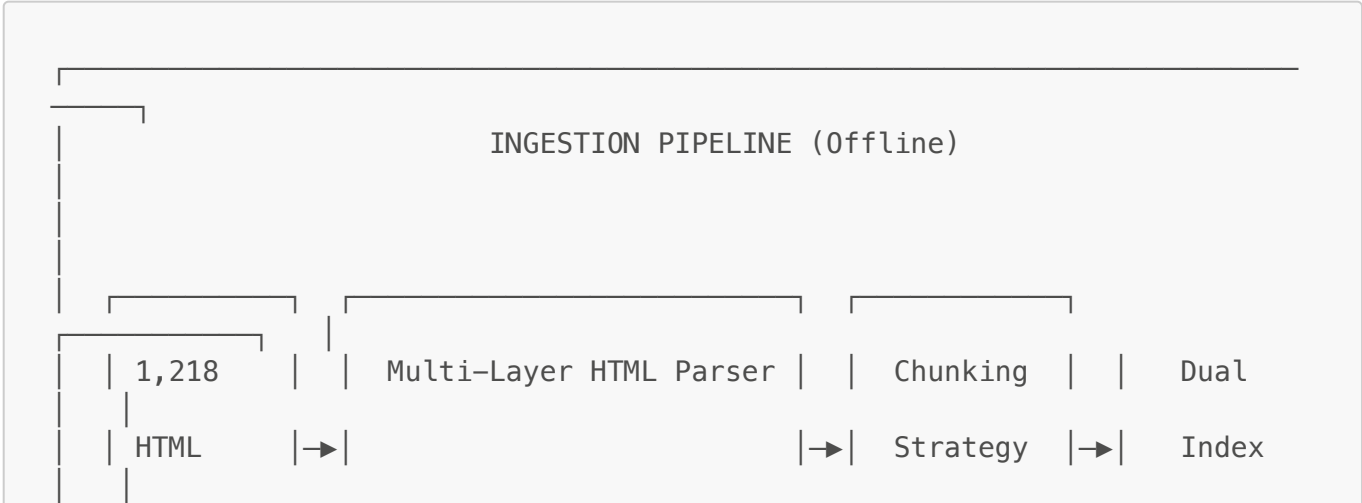
Memory check before starting (Apple Silicon):

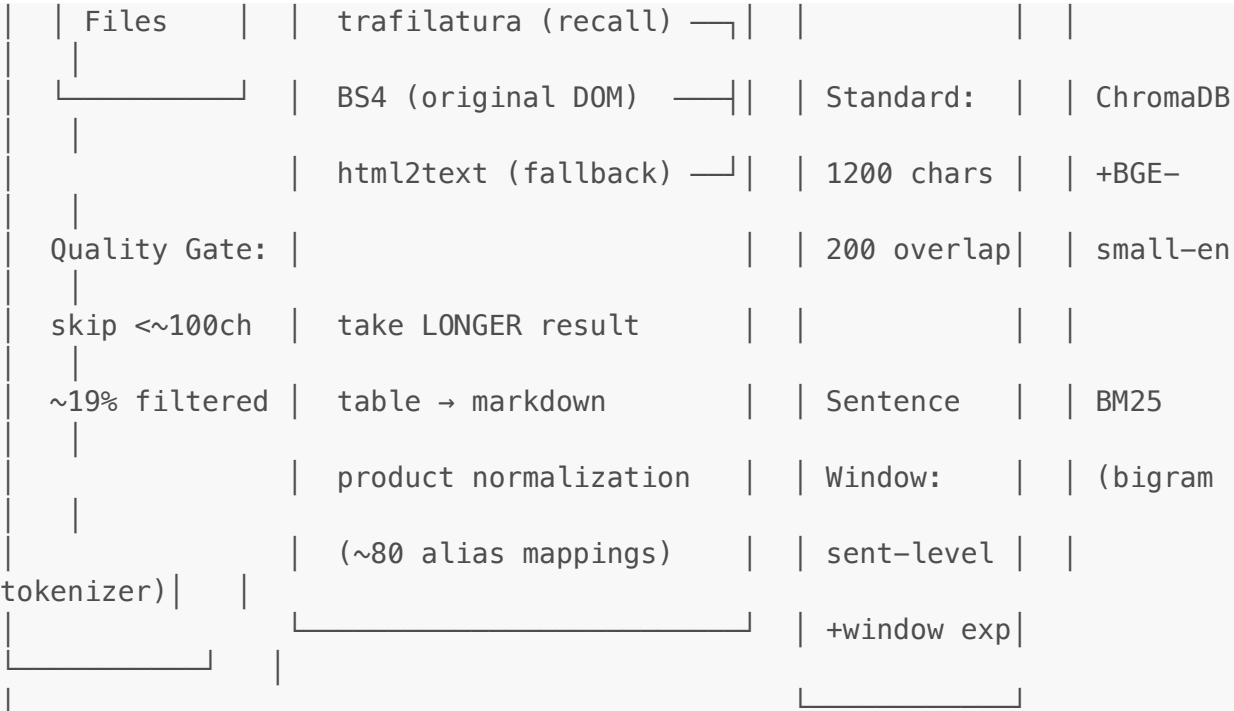
```
python3 -c "import psutil; m = psutil.virtual_memory(); print(f'Free: {m.available/1024**3:.1f} GB')"
```

Need ≥4.2 GB free for `gemma3:4b`

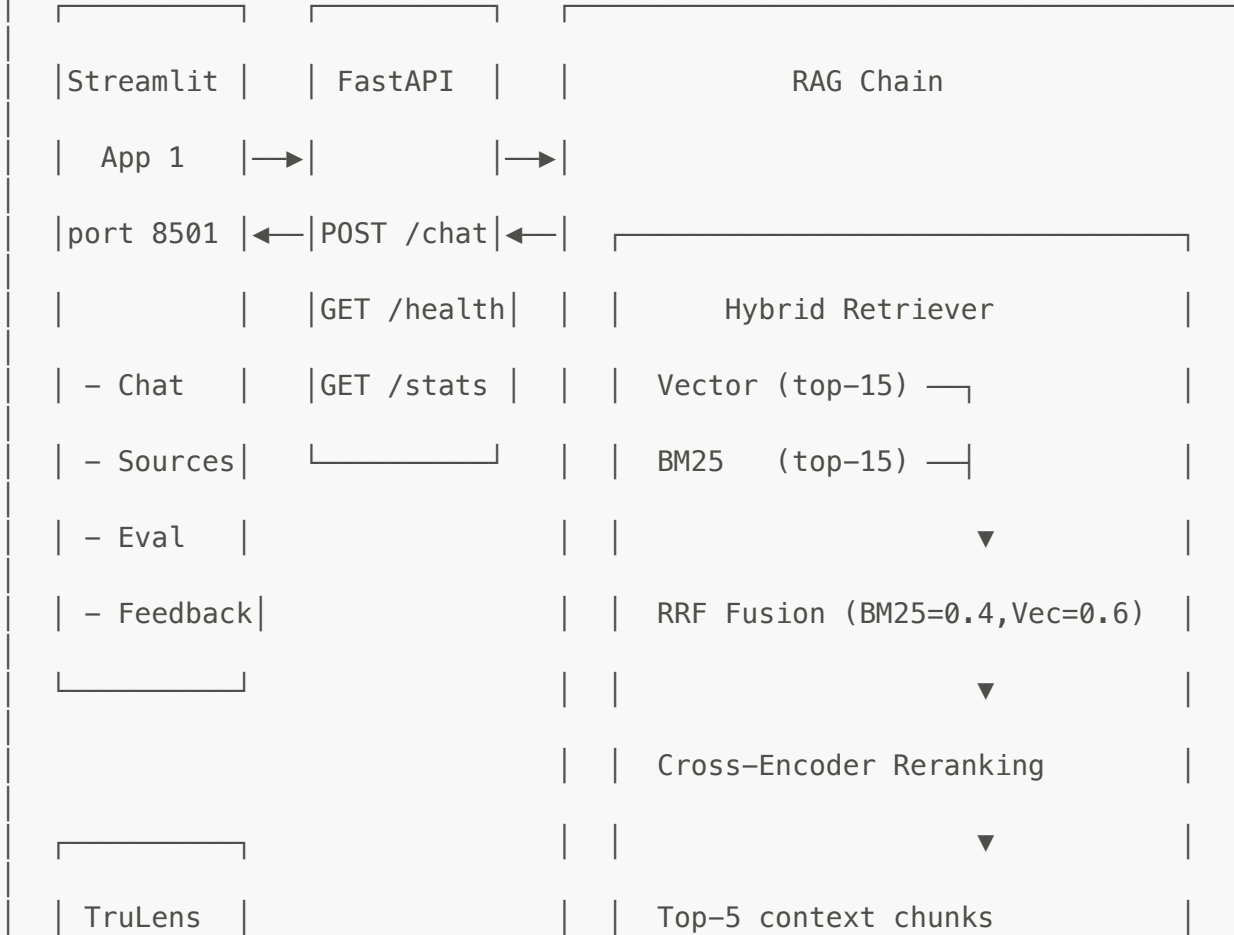
```
ollama ps # see what Ollama is holding in memory
ollama stop <model> # free a loaded model if needed
```

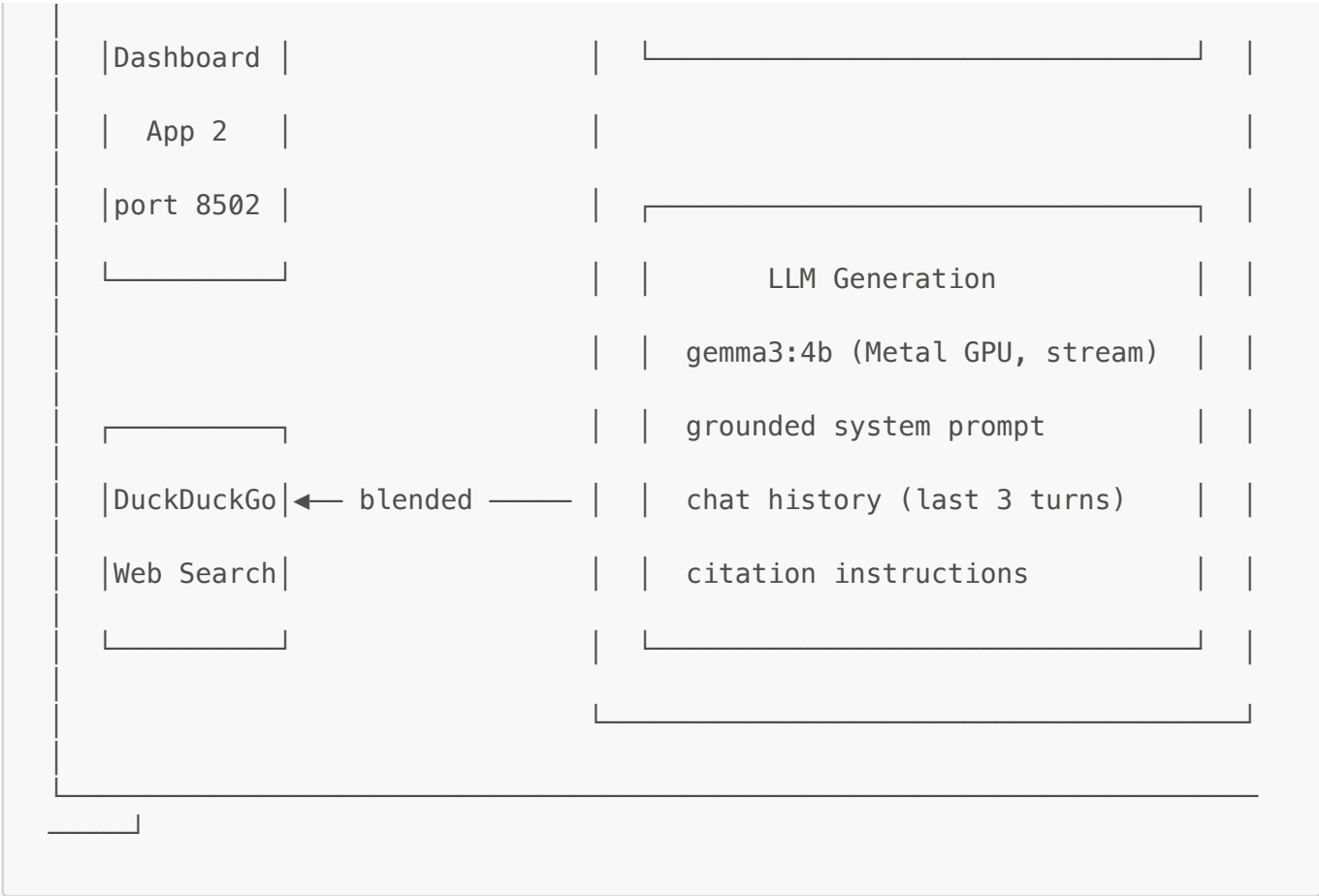
System Architecture





QUERY PIPELINE (Online)





Tech Stack

Component	Choice	Rationale
LLM	gemma3:4b via Ollama	Best speed/quality on Apple M-series Metal GPU (~4GB, 30–60 tok/s). Fully local — no data leaves the machine.
Embeddings	BAAI/bge-small-en-v1.5	Top MTEB benchmark score for its size (33M params). Outperforms all-MiniLM-L6-v2. Fast on CPU.
Vector DB	ChromaDB	Embedded, persistent, no server required. Correct for PoC. Production: Qdrant or Weaviate.
Keyword Search	BM25 (rank_bm25)	Catches exact product names/versions that dense embeddings miss. Bigram tokenizer so "AutoCAD LT" and "Fusion 360" match as compound units.
Reranker	cross-encoder/ms-marco-MiniLM-L-6-v2	Jointly encodes (query, passage) — far higher precision than bi-encoder similarity.
Fusion	Reciprocal Rank Fusion (RRF)	Merges vector + BM25 without score normalization. Standard in hybrid search (Cormack et al., 2009).

Component	Choice	Rationale
Evaluation	TruLens RAG Triad + LiteLLM/Ollama	LLM-as-judge. Post-hoc scoring decouples generation latency from eval latency.
Web Search	DuckDuckGo (ddgs)	No API key. Degrades gracefully if unavailable.
API	FastAPI	Async, auto-docs, Pydantic validation. StaticFiles at /files/ serves source HTML for clickable citations.
UI	Streamlit (two-app)	App 1 (8501): customer-facing. App 2 (8502): native TruLens admin. Zero coupling.
Package Mgmt	UV	10–100× faster than pip. Deterministic resolution via uv.lock .
Containerization	Docker + docker-compose	One-command deployment. Ollama + FastAPI + Streamlit as independent services.

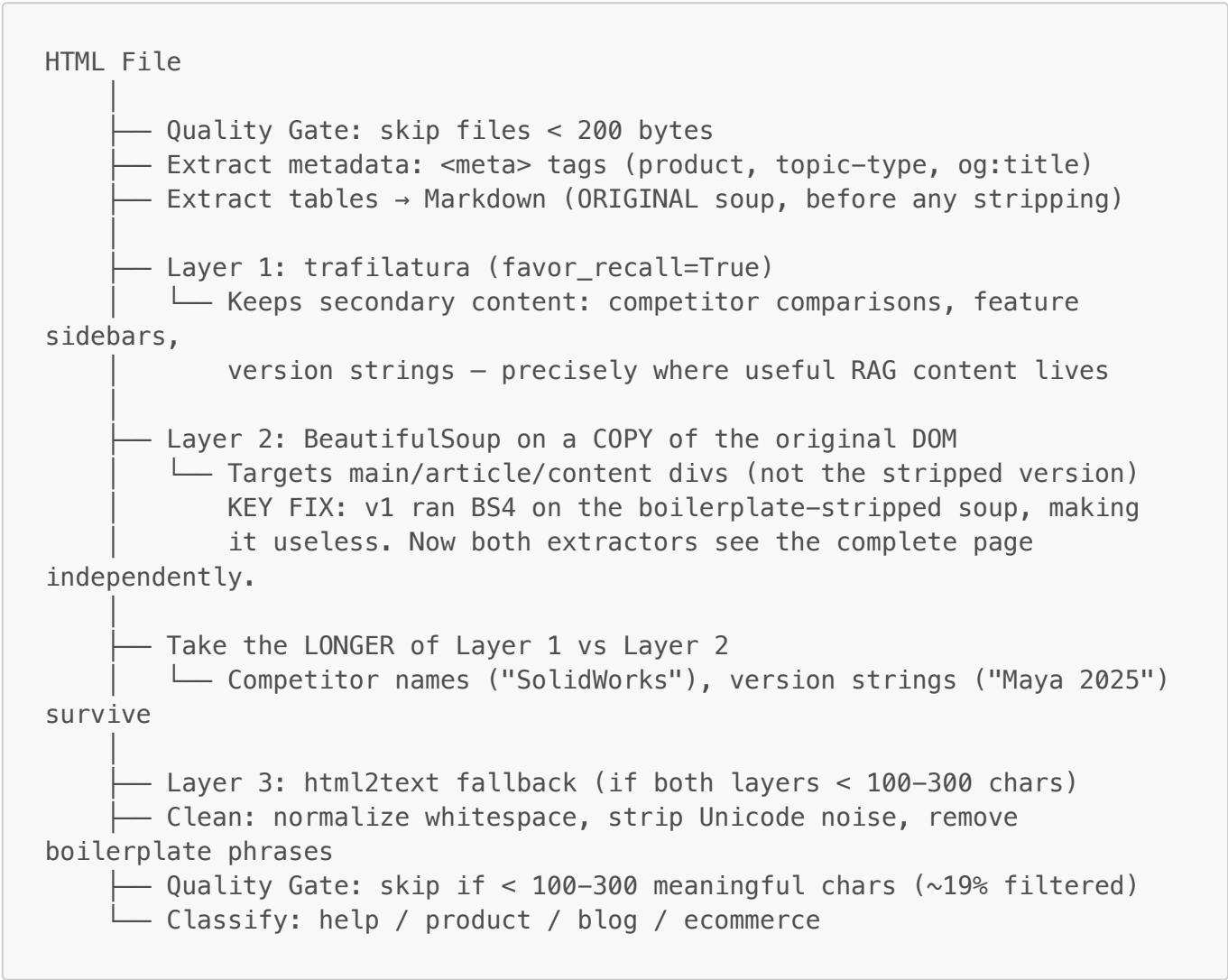
Assignment Coverage

Requirement	Implementation
Task 1a — corpus-only responses	<code>--mode corpus</code> — answers from ingested HTML only
Task 1b — blended web search responses	<code>--mode blended</code> — corpus + DuckDuckGo results
Evaluation tool with documented methodology	TruLens RAG Triad (Context Relevance, Groundedness, Answer Relevance)
Conversation / follow-up questions	Last 3 turns in prompt — coreference resolution tested
Source transparency	Per-message source expanders with clickable HTTP links
Hallucination mitigation	Grounded prompting + Groundedness scorer + cross-encoder reranking
Irrelevant question handling	Hard refusal instruction + irrelevant eval category
Human feedback	Thumbs up/down per response in Streamlit UI
Text pre-processing documented	Multi-layer parser, product aliases, data-driven chunk sizing
Document selection rationale	Quality filtering — all passing docs ingested, rationale documented
RAG architecture reflection	11-experiment iteration log with metric evidence
Evaluation validity argument	RAG Triad + category breakdown + negative results documented
requirements.txt / pyproject.toml	Both present

Requirement	Implementation
Sample questions answered	All 5 assignment questions (in UI) + 10 adversarial additions in eval suite
20-minute presentation	docs/Autodesk_RAG_Chatbot_Presentation.pptx
Production roadmap	"If I Had 1 Month" section below

Data Processing: Handling Noisy HTML

Multi-Layer Extraction Pipeline



Document Selection Rationale

All documents passing quality filtering are ingested — no manual curation. With 1,218 files, storage is not a bottleneck. Broad coverage is more valuable than narrow precision: false "I don't know" responses caused by missing documents are worse than occasional noise handled by the reranker.

Product Name Normalization

~80 internal Autodesk codes mapped to canonical names via [PRODUCT_ALIASES](#):

ACDIST → AutoCAD F360 → Fusion 360 ACAD_E → AutoCAD Electrical
RVIT → Revit ARCHDESK → AutoCAD Architecture NAVSIM → Navisworks
Simulate

Without this, BM25 cannot match "F360" against a user query for "Fusion 360", and embeddings see an unknown token rather than a known product name.

Data-Driven Chunk Sizing

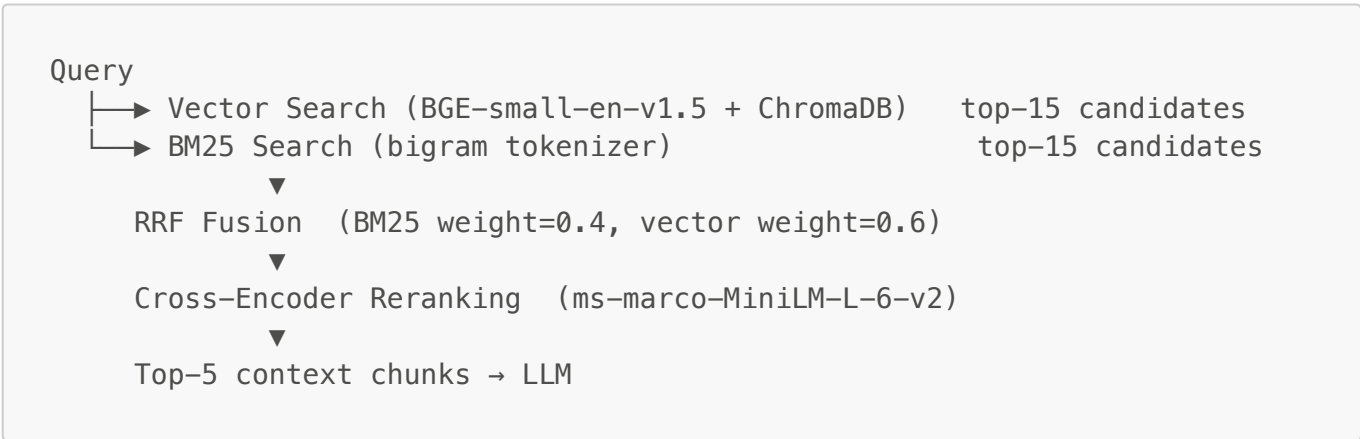
Corpus EDA ([scripts/analyze_data.py](#)) drove the chunk size decision:

Corpus statistics (983 docs after quality filtering):
Median: 1,235 chars P75: 2,666 P90: 5,877

Config	Chunks	Problem
chunk_size=512, overlap=64 (baseline)	9,266	Hard wall at ~600 chars; median doc splits into 3 fragments too small to carry a coherent answer
chunk_size=1200, overlap=200 (optimized)	~5,000	~50% of docs stay as single chunk — ideal RAG scenario

Retrieval Strategy

Standard Hybrid Retrieval



BM25 weight raised to 0.4 and candidate pool widened to 15 based on evaluation — this configuration produced the best blended composite score (0.826) across all experiments.

Sentence Window Retrieval (Experimental — Experiment 06)

A second retrieval strategy was implemented and evaluated to target Context Relevance improvement.

Architecture: Index at sentence granularity (~100–150 chars per chunk). At retrieval time, expand each retrieved sentence to its surrounding ±2 sentence window before passing to the LLM. Rationale: precise retrieval units with sufficient context for generation.

Result: Sentence window produced **lower** Context Relevance (0.448) than standard chunking (0.564) — a documented negative result. Explanation: sentence-level units are too fine-grained for the RAG Triad judge LLM. A 120-char sentence without surrounding context is harder to score as "relevant" than a 1,200-char chunk containing the same sentence plus its explanation. The correct path to higher CR is a stronger reranker or embedding model, not smaller retrieval granularity. This finding is preserved as Experiment 06 and documented in the metrics analysis plots.

Evaluation Framework

Methodology: The RAG Triad

Dimension	What it measures	Low score indicates
Context Relevance	Are retrieved chunks relevant to the query?	Retrieval failure — wrong chunks surfaced
Groundedness	Is every answer claim supported by context?	Hallucination — model invented content
Answer Relevance	Does the answer address what was asked?	Off-topic generation

Composite = geometric mean of all three. A single weak leg collapses the composite.

Why LLM-as-judge: Keyword overlap measures surface features, not semantic correctness. The RAG Triad is traceable to the RAGAS paper (Es et al., 2023), validated against human judgments, and suitable as a CI regression gate.

Test Suite

15 questions across 9 categories in `data/eval/eval_questions.json` — decoupled from code so non-engineers can add questions without touching Python:

Category	Example	What it probes
product_info	"What does Fusion 360 do?"	General knowledge retrieval
comparison	"Difference between AutoCAD and Revit?"	Multi-document synthesis
feature_query	"Does AutoCAD LT do 3D?"	Specific capability lookup
compatibility	"Can I use Fusion 360 on a Mac?"	Platform information
version_info	"Latest release for Maya?"	Version-specific recall
pricing_adversarial	"Exact annual price of Fusion 360 in USD?"	Hallucination probe — no exact price in corpus
version_adversarial	"AutoCAD version released in 2024?"	Hallucination probe — must not invent

Category	Example	What it probes
recency_probe	"Newest AI features in 2025?"	Corpus coverage limit — blended advantage
irrelevant	"Tell me about quantum computing"	Graceful refusal

Post-Hoc Scoring Architecture

```
for each question:
  1. RAG pipeline runs → answer + context (corpus + web chunks via cache)
  2. TruBasicApp records   Input = question, Output = answer

after all questions:
  3. Judge LLM scores all (question, answer, context) triples
  4. FeedbackResult rows batch-written to TruLens DB (keyed by record_id)
  5. Timestamped JSON exported to data/processed/experiment_results/
```

This decouples generation latency from judge latency — the correct pattern for production async evaluation. **Critical fix:** the eval loop previously re-ran `retriever.retrieve()` for context, making blended mode invisible to the scorer (web chunks not returned by retrieval-only calls). Fix: cache the full `RAGResponse` inside the generation callable and extract `response.sources` directly.

Two-App Dashboard Architecture

	App 1 — Stakeholder	App 2 — MLE Admin
Port	8501	8502
Start	<code>streamlit run ui/app.py</code>	<code>python scripts/run_admin_dashboard.py</code>
Data	JSON exports (no TruLens import, <1s startup)	<code>trulens.sqlite</code> (native TruLens session)
Shows	RAG Triad scores, radar chart, corpus vs blended	Per-query traces, CoT reasoning, leaderboard

Experiment Results

10 experiments iterated on ingestion, retrieval, and generation parameters:

Experiment	Key Change	Corpus Composite	Blended Composite
01_baseline	chunk=512, precision mode, no aliases	0.689	0.819

Experiment	Key Change	Corpus Composite	Blended Composite
02_optimized_html_parser	chunk=1200, recall mode, ~80 aliases	0.718	0.815
03_optimized_MIN_TEXT_LENGTH	min_text threshold raised to 400	0.710	0.758
04_retrieval_weights	top_k=15, BM25=0.4	0.710	0.826 ← best blended
05_quality_filter	ecommerce pages excluded	0.760 ← best corpus	0.796
06_sentence_window	sentence-level chunks (negative result)	0.591	0.758
07_prompt_engineering	structured output hints, hard refusal	0.705	0.731
08–10	combined filter + weight variants	~0.710	—

Key finding: Context Relevance is the persistent weak leg (avg ~0.56). Groundedness is consistently strong (avg ~0.97), confirming the grounded prompting approach is effective at hallucination prevention. CR is structurally limited by three categories: **irrelevant** (correct refusal behavior), **recency_probe** (static corpus limitation), and **comparison** (multi-document synthesis). Removing these from the aggregate shifts corpus CR from 0.56 to ~0.65.

Running Evaluations

```
# Standard evaluation run (skips ingestion if index already populated)
uv run python scripts/run_trulens_eval.py \
  --mode both \
  --experiment "05_quality_filter" \
  --pipeline optimized

# Baseline experiment (builds separate baseline ChromaDB collection)
uv run python scripts/run_trulens_eval.py \
  --mode both \
  --experiment "01_baseline" \
  --pipeline baseline \
  --force-ingest

# Sentence window experiment
uv run python scripts/run_trulens_eval.py \
  --mode both \
  --experiment "06_sentence_window" \
  --pipeline sentence_window \
  --force-ingest
```

```
# Open TruLens admin dashboard
uv run python scripts/run_admin_dashboard.py      # → http://localhost:8502

# Generate experiment analysis plots (PNG)
uv run python scripts/analyze_metrics.py

# Generate corpus EDA plots (PNG)
uv run python scripts/analyze_data.py
```

Flag	Effect
<code>--experiment NAME</code>	Sets <code>app_name</code> in TruLens Leaderboard
<code>--mode corpus/blended/both</code>	Sets <code>app_version</code> — corpus vs blended as comparable versions
<code>--pipeline optimized/baseline/sentence_window</code>	Selects ingestion stack and ChromaDB collection
<code>--force-ingest</code>	Forces full re-parse → chunk → embed
<code>--reset-db</code>	Wipes all TruLens records — use deliberately

Project Structure

```
autodesk-rag-chatbot/
├── data/
│   ├── raw/                # Place HTML files here (provided by
Autodesk)
│   ├── eval/
│   │   └── eval_questions.json  # 15-question test suite (in git)
│   ├── processed/
│   │   ├── experiment_results/  # 11 experiment JSON results (in git)
│   │   ├── corpus_analysis/     # EDA PNG plots (in git)
│   │   ├── metrics_analysis/    # Experiment PNG plots (in git)
│   │   └── trulens.sqlite       # TruLens evaluation DB (in git)
│   └── chroma_db/             # Placeholder – populated by
ingestion
├── src/
│   ├── config.py             # Centralised settings
│   ├── ingestion/
│   │   └── html_parser.py      # Optimized: trafilatura recall + BS4
original DOM
│   ├── html_parser_baseline.py # Baseline parser (Exp 01 – bugs
preserved intentionally)
│   ├── chunker.py            # Optimized: 1200 chars, tables
merged, prefix budget
│   ├── chunker_baseline.py    # Baseline chunker (Exp 01)
│   ├── pipeline.py            # Ingestion with fast-path skip
│   └── pipeline_baseline.py   # Baseline ingestion (isolated
```

```

ChromaDB collection)
|   |   └─ pipeline_sentence_window.py  # Sentence window ingestion
|   └─ retrieval/
|       └─ vector_store.py             # ChromaDB + BGE-small-en-v1.5
|       └─ bm25_retriever.py           # BM25Okapi with bigram tokenizer
|       └─ hybrid_retriever.py         # RRF fusion + cross-encoder
reranking
|   |   └─ sentence_window_retriever.py # Sentence retrieval + window
expansion
|   |   └─ web_search.py                # DuckDuckGo (blended mode)
|   └─ generation/
|       └─ llm.py                      # Ollama client (streaming, Metal
GPU, warmup)
|   |   └─ prompts.py                  # Grounded templates + per-category
format hints
|   |   └─ rag_chain.py                 # RAG orchestrator (corpus + blended)
|   └─ evaluation/
|       └─ db.py                      # TruLens DB path + session factory
|       └─ evaluator.py                # RAG Triad (post-hoc scoring,
context cache fix)
|   └─ api/
|       └─ main.py                    # FastAPI (/chat, /health, /stats,
/files)
└─ ui/
    └─ app.py                        # Streamlit App 1: chat + eval
summary
└─ scripts/
    └─ ingest.py                    # Standalone ingestion script
    └─ run_trulens_eval.py          # Evaluation runner (all pipeline
variants)
    └─ run_admin_dashboard.py        # TruLens App 2 (port 8502)
    └─ analyze_data.py              # Corpus EDA → PNG plots
    └─ analyze_metrics.py           # Experiment metrics → PNG plots
    └─ build_presentation.py         # Generates PPTX
└─ tests/
    └─ test_core.py                 # 24 unit tests
└─ docs/
    └─ Autodesk_RAG_Chatbot_Presentation.pptx
└─ Dockerfile
└─ docker-compose.yml
└─ Makefile
└─ pyproject.toml
└─ requirements.txt
└─ .env.example
└─ README.md

```

Running Tests

```

uv run python -m pytest tests/ -v
# 24 passed in ~6s

```

Tests cover: HTML parser (empty files, JS shells, valid pages, alias normalization), chunker (single-chunk optimization, table merging), BM25 (compound product name matching via bigram tokenizer), prompts (context formatting, history), evaluation (question loader, score unpacking for all three provider return formats).

Known Limitations

- **Ollama must be running:** Start with `ollama serve` in a dedicated terminal before launching the API. On Mac, Ollama can also be started via the menu bar app if installed — in that case `ollama serve` is not needed as it runs automatically in the background.
 - **Port conflict between Docker and local dev:** Both modes use port `11434` for Ollama. Stop one before starting the other — `docker-compose down` before running locally, or `pkill ollama` before running Docker.
 - **Same-model judge bias:** `gemma3:4b` judges its own outputs. A stronger independent model (GPT-4o, Claude 3.5 Sonnet) would eliminate this. One-line change in `evaluator.py`.
 - **Static corpus:** Fixed crawl date. Recency probes ("2025 AI features") score lower on corpus mode by design — blended mode partially mitigates this.
 - **Context Relevance ceiling:** Structurally limited by `irrelevant` (correct refusal), `recency_probe` (corpus gap), and `comparison` (multi-document challenge) categories. Removing these shifts corpus CR from 0.56 to ~0.65.
 - **Source rendering:** Locally-served HTML opens without Autodesk CDN CSS. Content is identical. Production fix: store canonical `autodesk.com` URLs.
 - **TruLens OTEL constraint:** TruLens 2.7.x locks record mutation APIs with OTEL active. Per-question metadata stored in JSON exports instead of TruLens record tags.
-

If I Had 1 Month (Production Roadmap)

Week 1: Data & Infrastructure

- ☐ Full re-crawl with Playwright to capture JS-rendered content
- ☐ PostgreSQL for conversation history and feedback storage
- ☐ ChromaDB → Qdrant for production-grade vector search

Week 2: Retrieval Quality

- ☐ Embedding upgrade: `bge-small` → `bge-large-en-v1.5` (expected +5 CR points)
- ☐ Reranker upgrade: `MiniLM-L-6` → `MiniLM-L-12` (no re-ingest needed)
- ☐ Query understanding: intent classification + entity extraction before retrieval
- ☐ Auto-merging retrieval: hierarchical parent/child chunks for comparison queries

Week 3: Generation & Agent Features

- ☐ LangGraph multi-agent: Router → Retrieval → Web search → Citation validator
- ☐ Tool-use for structured queries (pricing API, product catalog)
- ☐ Stronger judge model (GPT-4o) to eliminate same-model evaluation bias

Week 4: Evaluation & Deployment

- ☒ LLM-as-judge evaluation — implemented via TruLens RAG Triad
 - ☐ CI evaluation gate: automatic regression testing on every config change
 - ☐ Kubernetes deployment with auto-scaling
 - ☐ Monitoring: latency P99, error rates, per-category hallucination tracking
 - ☐ Collect and Store users feedback, analyze and act on
-

License

This project was created as part of Autodesk's interview process. All materials are confidential and the property of Autodesk. Do not redistribute.